

Ejercicio 3: Modelo Vectorial y TF-IDF

Objetivo de la práctica

- Comprender el modelo vectorial como base para representar documentos y consultas.
- Calcular la matriz TF-IDF para el corpus data/01_corpus_turismo_500.txt
- Calcular la matriz TF-IDF para el corpus Gutenberg 1000

Paso 1: Calcular la matriz TF-IDF para el corpus data/01_corpus_turismo_500.txt

Paso 1: Cargar el corpus

```
import os
import math
import re
import string
from collections import Counter, defaultdict

# Cargar corpus de turismo
ruta_turismo = os.path.join(os.getcwd(), '01_corpus_turismo_500.txt')

with open(ruta_turismo, 'r', encoding='utf-8') as f:
    corpus_turismo = f.read().splitlines()

print(f'Número de documentos: {len(corpus_turismo)}')
print(f'Ejemplo de documento (sin limpiar):')
print(corpus_turismo[0][:200])
```

Número de documentos: 500

Ejemplo de documento (sin limpiar):

Otavaló es conocido por su mercado indígena y su artesanía Perfecto para rafting.

Paso 2: Limpieza del texto

Antes de tokenizar, se debe limpiar cada documento:

1. **Convertir a minúsculas** — para que “Casa” y “casa” sean la misma palabra.
2. **Eliminar signos de puntuación** — comas, puntos, paréntesis, etc.
3. **Eliminar números** (opcional) — depende del dominio.
4. **Eliminar stopwords** — palabras muy comunes que no aportan significado (“de”, “la”, “el”, “en”, etc.).

```
STOPWORDS_ES = {
    'de', 'la', 'que', 'el', 'en', 'y', 'a', 'los', 'del', 'se',
    'las', 'por', 'un', 'para', 'con', 'no', 'una', 'su', 'al',
    'lo', 'como', 'más', 'mas', 'pero', 'sus', 'le', 'ya', 'o',
    'este', 'sí', 'si', 'porque', 'esta', 'entre', 'cuando', 'muy',
    'sin', 'sobre', 'también', 'me', 'hasta', 'hay', 'donde', 'quien',
    'desde', 'todo', 'nos', 'durante', 'todos', 'uno', 'les', 'ni',
    'contra', 'otros', 'ese', 'eso', 'ante', 'ellos', 'e', 'esto',
    'mi', 'antes', 'algunos', 'qué', 'unos', 'yo', 'otro', 'otras',
    'otra', 'él', 'tanto', 'esa', 'estos', 'mucho', 'quienes',
    'nada', 'muchos', 'cual', 'poco', 'ella', 'estar', 'estas',
    'algunas', 'algo', 'nosotros', 'fue', 'son', 'es', 'ha', 'era',
    'ser', 'tiene', 'han', 'ser', 'son', 'sido', 'tiene', 'cada',
    'donde', 'dos', 'puede', 'aquí', 'así', 'asi', 'tan', 'cual',
    'vez', 'sea', 'sus', 'solo', 'solo', 'después', 'parte', 'esas'
}

print(f'Stopwords cargadas: {len(STOPWORDS_ES)} palabras')
print(f'Ejemplos: {list(STOPWORDS_ES)[:15]}')
```

Stopwords cargadas: 107 palabras

Ejemplos: ['fue', 'dos', 'estar', 'otro', 'este', 'solo', 'son', 'si', 'muy', 'mi', 'quienes']

```
def limpiar_texto(texto):
    """
    Limpia un texto:
    1. Convierte a minúsculas
    2. Elimina signos de puntuación y caracteres especiales
    3. Elimina números
    4. Elimina espacios extra
    """
    texto = texto.lower()
```

```

# Eliminar puntuación y caracteres especiales (dejar solo letras y espacios)
texto = re.sub(r'[~a-záéíóúüñ\s]', '', texto)

# Eliminar espacios múltiples
texto = re.sub(r'\s+', ' ', texto).strip()

return texto

# Probar con un ejemplo
ejemplo = corpus_turismo[0]
print("ORIGINAL:")
print(ejemplo[:200])
print()
print("LIMPIO:")
print(limpiar_texto(ejemplo)[:200])

```

ORIGINAL:

Otavaló es conocido por su mercado indígena y su artesanía Perfecto para rafting.

LIMPIO:

otavaló es conocido por su mercado indígena y su artesanía perfecto para rafting

Paso 3: Tokenización

Tokenizar = dividir el texto limpio en una lista de palabras individuales (tokens). Además, eliminamos las stopwords en este paso.

```

def tokenizar(texto, stopwords=STOPWORDS_ES):
    """
    Tokeniza un texto limpio:
    1. Divide por espacios
    2. Elimina stopwords
    3. Elimina tokens de 1 o 2 caracteres (poco informativos)
    Retorna una lista de tokens.
    """
    # Primero limpiar
    texto_limpio = limpiar_texto(texto)

    # Dividir en palabras

```

```

tokens = texto_limpio.split()

# Filtrar stopwords y palabras muy cortas
tokens = [t for t in tokens if t not in stopwords and len(t) > 2]

return tokens

# Probar con un ejemplo
ejemplo = corpus_turismo[0]
tokens_ejemplo = tokenizar(ejemplo)
print(f"Documento original ({len(ejemplo)} caracteres):")
print(ejemplo[:150], "...")
print()
print(f"Tokens resultantes ({len(tokens_ejemplo)} tokens):")
print(tokens_ejemplo[:20])

```

Documento original (81 caracteres):

Otavales es conocido por su mercado indígena y su artesanía Perfecto para rafting. ...

Tokens resultantes (7 tokens):

['otavale', 'conocido', 'mercado', 'indígena', 'artesanía', 'perfecto', 'rafting']

```

# Tokenizar TODO el corpus
corpus_tokenizado = [tokenizar(doc) for doc in corpus_turismo]

print(f"Documentos tokenizados: {len(corpus_tokenizado)}")

# Estadísticas de tokens
total_tokens = sum(len(doc) for doc in corpus_tokenizado)
print(f"Total de tokens en el corpus: {total_tokens:,}")
print(f"Promedio de tokens por documento: {total_tokens / len(corpus_tokenizado):.1f}")

```

Documentos tokenizados: 500

Total de tokens en el corpus: 3,742

Promedio de tokens por documento: 7.5

Paso 4: Construir el vocabulario

El vocabulario es el conjunto de todas las palabras únicas del corpus. Cada palabra se mapea a un índice (columna de la matriz).

```

def construir_vocabulario(corpus_tokenizado, min_df=2, max_df_ratio=0.95):
    """
    Construye el vocabulario a partir del corpus tokenizado.
    Args:
        corpus_tokenizado: lista de listas de tokens
        min_df: mínimo de documentos en que debe aparecer un término
        max_df_ratio: máximo porcentaje de documentos (0.95 = 95%)

    Returns:
        vocab: dict {palabra: índice}
        vocab_lista: lista ordenada de palabras
    """
    N = len(corpus_tokenizado)
    max_df = int(max_df_ratio * N)

    # Contar en cuántos documentos aparece cada término (Document Frequency)
    df_counter = Counter()
    for doc_tokens in corpus_tokenizado:
        # set() para contar cada término UNA sola vez por documento
        palabras_unicas = set(doc_tokens)
        df_counter.update(palabras_unicas)

    # Filtrar: mantener solo términos con min_df <= df <= max_df
    vocabulario_filtrado = sorted([
        palabra for palabra, freq in df_counter.items()
        if min_df <= freq <= max_df
    ])

    # Crear diccionario {palabra: índice}
    vocab = {palabra: idx for idx, palabra in enumerate(vocabulario_filtrado)}

    print(f"Palabras únicas totales: {len(df_counter)}")
    print(f"Palabras después de filtrar (min_df={min_df}, max_df={max_df}): {len(vocab)}")

    return vocab, vocabulario_filtrado

vocab, vocab_lista = construir_vocabulario(corpus_tokenizado, min_df=2, max_df_ratio=0.95)
print(f"\nPrimeras 20 palabras del vocabulario:")
print(vocab_lista[:20])

```

Palabras únicas totales: 95

Palabras después de filtrar (min_df=2, max_df=475): 95

Primeras 20 palabras del vocabulario:

['agua', 'amazonía', 'arquitectura', 'artesanía', 'atrae', 'atraen', 'auténtico', 'aventura',

Paso 5: Construir la Matriz de Frecuencia de Términos (TF)

$TF(t, d)$ = número de veces que el término t aparece en el documento d .

La matriz tiene forma (N_documentos × N_vocabulario).

```
import numpy as np

def construir_matriz_tf(corpus_tokenizado, vocab):
    """
    Construye la matriz TF (Term Frequency).

    Cada celda [i][j] = frecuencia del término j en el documento i.

    Returns:
        matriz_tf: numpy array de forma (N_docs, N_vocab)
    """
    N_docs = len(corpus_tokenizado)
    N_vocab = len(vocab)

    # Inicializar matriz de ceros
    matriz_tf = np.zeros((N_docs, N_vocab), dtype=np.float64)

    for i, doc_tokens in enumerate(corpus_tokenizado):
        # Contar frecuencia de cada token en el documento
        conteo = Counter(doc_tokens)

        for palabra, frecuencia in conteo.items():
            if palabra in vocab:
                j = vocab[palabra]
                matriz_tf[i][j] = frecuencia

    return matriz_tf

matriz_tf = construir_matriz_tf(corpus_tokenizado, vocab)
```

```

print(f"Forma de la matriz TF: {matriz_tf.shape}")
print(f" → {matriz_tf.shape[0]} documentos × {matriz_tf.shape[1]} términos")
print()

# Mostrar un fragmento
print("Fragmento de la matriz TF (docs 0-4, términos 0-9):")
print(f"Términos: {vocab_lista[:10]}")
print(matriz_tf[:5, :10])

```

Forma de la matriz TF: (500, 95)
 → 500 documentos × 95 términos

Fragmento de la matriz TF (docs 0-4, términos 0-9):
 Términos: ['agua', 'amazonía', 'arquitectura', 'artesanía', 'atrae', 'atraen', 'auténtico',
 [[0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

Paso 6: Construir el vector DF (Document Frequency) y calcular IDF

- $DF(t)$ = número de documentos en los que aparece el término t .
- $IDF(t) = \log(N / DF(t))$ donde N = total de documentos.

```

def construir_df_idf(matriz_tf):
    """
    Calcula DF e IDF a partir de la matriz TF.

    DF(t) = número de documentos donde TF(t,d) > 0
    IDF(t) = log(N / DF(t))

    Returns:
        vector_df: array de forma (N_vocab,)
        vector_idf: array de forma (N_vocab,)
    """
    N_docs = matriz_tf.shape[0]

    # DF: contar documentos donde el término aparece (TF > 0)
    vector_df = np.sum(matriz_tf > 0, axis=0) # sumar por columnas

```

```

# IDF: log(N / DF)
# Usamos np.where para evitar división por cero (no debería pasar si min_df >= 1)
vector_idf = np.log(N_docs / np.where(vector_df > 0, vector_df, 1))

return vector_df, vector_idf

vector_df, vector_idf = construir_df_idf(matriz_tf)

print(f"Vector DF - forma: {vector_df.shape}")
print(f"Vector IDF - forma: {vector_idf.shape}")
print()

# Mostrar términos con mayor y menor IDF
print("Términos con MAYOR IDF (raros, discriminativos):")
top_idf = np.argsort(vector_idf)[::-1][:10]
for idx in top_idf:
    print(f" {vocab_lista[idx]:<25} DF={int(vector_df[idx]):>3} IDF={vector_idf[idx]:.4f}")

print()
print("Términos con MENOR IDF (comunes, poco discriminativos):")
bottom_idf = np.argsort(vector_idf)[:10]
for idx in bottom_idf:
    print(f" {vocab_lista[idx]:<25} DF={int(vector_df[idx]):>3} IDF={vector_idf[idx]:.4f}")

```

Vector DF - forma: (95,)
Vector IDF - forma: (95,)

Términos con MAYOR IDF (raros, discriminativos):

increíble	DF= 11	IDF=3.8167
auténtico	DF= 13	IDF=3.6497
sorprendente	DF= 15	IDF=3.5066
tranquilo	DF= 16	IDF=3.4420
surf	DF= 17	IDF=3.3814
humanidad	DF= 20	IDF=3.2189
centro	DF= 20	IDF=3.2189
quito	DF= 20	IDF=3.2189
espectacular	DF= 20	IDF=3.2189
histórico	DF= 20	IDF=3.2189

Términos con MENOR IDF (comunes, poco discriminativos):

ideal	DF=142	IDF=1.2588
-------	--------	------------

feriado	DF=138	IDF=1.2874
próximo	DF=118	IDF=1.4439
perfecto	DF=100	IDF=1.6094
experiencia	DF= 96	IDF=1.6503
inolvidable	DF= 96	IDF=1.6503
lugar	DF= 92	IDF=1.6928
visitar	DF= 92	IDF=1.6928
montañita	DF= 74	IDF=1.9105
gastronomía	DF= 66	IDF=2.0250

Paso 7: Construir la Matriz TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Simplemente multiplicamos cada fila de la matriz TF por el vector IDF.

```
def construir_matriz_tfidf(matriz_tf, vector_idf):
    """
    Calcula la matriz TF-IDF.

    TF-IDF(t, d) = TF(t, d) * IDF(t)

    Multiplicamos cada columna de la matriz TF por su IDF correspondiente.
    """
    # Multiplicación elemento a elemento: cada fila se multiplica por el vector IDF
    matriz_tfidf = matriz_tf * vector_idf

    return matriz_tfidf

matriz_tfidf = construir_matriz_tfidf(matriz_tf, vector_idf)

print(f"Forma de la matriz TF-IDF: {matriz_tfidf.shape}")
print(f" → {matriz_tfidf.shape[0]} documentos × {matriz_tfidf.shape[1]} términos")

# Densidad: qué porcentaje de la matriz está "llena"
total_celdas = matriz_tfidf.shape[0] * matriz_tfidf.shape[1]
no_cero = np.count_nonzero(matriz_tfidf)
print(f"Densidad (valores != 0): {no_cero / total_celdas:.2%}")
```

Forma de la matriz TF-IDF: (500, 95)
→ 500 documentos × 95 términos

Densidad (valores != 0): 7.83%

Paso 8: Explorar la matriz TF-IDF

```
import pandas as pd
df_tfidf = pd.DataFrame(matriz_tfidf, columns=vocab_lista)
print("Primeras 5 filas y 10 columnas de la matriz TF-IDF:")
df_tfidf.iloc[:5, :10]
```

Primeras 5 filas y 10 columnas de la matriz TF-IDF:

	agua	amazonía	arquitectura	artesanía	atrae	atraen	auténtico	aventura	aves	avistamiento
0	0.0	0.0	0.0	2.718101	0.000000	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.000000	2.918771	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0

```
# Top 15 términos por TF-IDF promedio
tfidf_promedio = df_tfidf.mean(axis=0).sort_values(ascending=False)

print("Top 15 términos por TF-IDF promedio (más discriminativos):")
for i, (termino, valor) in enumerate(tfidf_promedio.head(15).items()):
    print(f" {i+1:>2}. {termino:<25} {valor:.4f}")
```

Top 15 términos por TF-IDF promedio (más discriminativos):

1.	ideal	0.3877
2.	feriado	0.3733
3.	próximo	0.3408
4.	perfecto	0.3219
5.	experiencia	0.3168
6.	inolvidable	0.3168
7.	lugar	0.3115
8.	visitar	0.3115
9.	montañita	0.2828
10.	gastronomía	0.2673
11.	visitantes	0.2544
12.	rafting	0.2404

13. ruta	0.2223
14. spondylus	0.2223
15. famosas	0.2223

Paso 9: Similitud coseno y función buscar()

La similitud coseno mide el ángulo entre dos vectores:

$$\text{coseno}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \times \|\vec{b}\|}$$

Valor entre 0 (sin relación) y 1 (idénticos en contenido).

```
def similitud_coseno(vec_a, vec_b):
    """
    Calcula la similitud coseno entre dos vectores.

    coseno(a, b) = (a · b) / (||a|| * ||b||)
    """
    producto_punto = np.dot(vec_a, vec_b)
    norma_a = np.linalg.norm(vec_a)
    norma_b = np.linalg.norm(vec_b)

    if norma_a == 0 or norma_b == 0:
        return 0.0

    return producto_punto / (norma_a * norma_b)

def vectorizar_consulta(query, vocab, vector_idf):
    """
    Convierte una consulta de texto en un vector TF-IDF.
    Usa el MISMO vocabulario e IDF del corpus (no se re-entrena).
    """
    tokens = tokenizar(query)
    conteo = Counter(tokens)

    vector_tf = np.zeros(len(vocab))
    for palabra, freq in conteo.items():
        if palabra in vocab:
            vector_tf[vocab[palabra]] = freq
```

```

# Multiplicar por IDF
vector_tfidf = vector_tf * vector_idf
return vector_tfidf

def buscar(query, matriz_tfidf, vocab, vector_idf, nombres=None, top_k=5):
    """
    Busca los documentos más relevantes para una consulta.

    Args:
        query: texto de búsqueda
        matriz_tfidf: la matriz TF-IDF del corpus
        vocab: diccionario del vocabulario
        vector_idf: vector IDF del corpus
        nombres: lista de nombres de documentos (opcional)
        top_k: cuántos resultados devolver

    Returns:
        Lista de tuplas (índice_o_nombre, score)
    """
    # 1. Vectorizar la consulta
    vec_query = vectorizar_consulta(query, vocab, vector_idf)

    # 2. Calcular similitud coseno contra cada documento
    scores = np.array([
        similitud_coseno(vec_query, matriz_tfidf[i])
        for i in range(matriz_tfidf.shape[0])
    ])

    # 3. Ordenar de mayor a menor
    indices_ordenados = np.argsort(scores)[::-1][:top_k]

    # 4. Construir resultados
    resultados = []
    for idx in indices_ordenados:
        nombre = nombres[idx] if nombres else f"Documento {idx}"
        resultados.append((nombre, round(scores[idx], 4)))

    return resultados

```

```
# Probar la búsqueda sobre el corpus de turismo
consultas = [
    "arquitectura y sol",
    "naturaleza ecológico",
    "hotel restaurante",
]

for consulta in consultas:
    print(f'\nConsulta: "{consulta}"')
    resultados = buscar(consulta, matriz_tfidf, vocab, vector_idf, top_k=3)
    for nombre, score in resultados:
        # Mostrar fragmento del documento
        idx = int(nombre.replace("Documento ", ""))
        fragmento = corpus_turismo[idx][:200]
        print(f" [{score:.4f}] {nombre}: {fragmento}...")
```

Consulta: "arquitectura y sol"

[0.4679] Documento 220: Cuenca deslumbra con su arquitectura colonial y gastronomía...

[0.4679] Documento 92: Cuenca deslumbra con su arquitectura colonial y gastronomía...

[0.4679] Documento 452: Cuenca deslumbra con su arquitectura colonial y gastronomía...

Consulta: "naturaleza ecológico"

[0.3159] Documento 175: Vilcabamba atrae visitantes interesados en longevidad y naturaleza

[0.3159] Documento 185: Vilcabamba atrae visitantes interesados en longevidad y naturaleza

[0.3159] Documento 213: Vilcabamba atrae visitantes interesados en longevidad y naturaleza

Consulta: "hotel restaurante"

[0.0000] Documento 499: Baños de Agua Santa ofrece deportes de aventura como rafting Perfe

[0.0000] Documento 170: Mindo es famoso por el avistamiento de aves Un lugar increíble par

[0.0000] Documento 157: Vilcabamba atrae visitantes interesados en longevidad y naturaleza

Paso 10 (Opcional): Corpus Gutenberg 1000

El mismo proceso aplicado a los libros de Gutenberg. Se usa `sublinear_tf` ($\log(1 + TF)$) para compensar documentos muy largos.

```
# Cargar corpus Gutenberg
ruta_libros = os.path.join(os.getcwd(), '1000libros')
archivos = sorted(os.listdir(ruta_libros))
print(f'Libros disponibles: {len(archivos)}')
```

```

corpus_gutenberg = []
nombres_gutenberg = []

for archivo in archivos:
    ruta_archivo = os.path.join(ruta_libros, archivo)
    try:
        with open(ruta_archivo, 'r', encoding='utf-8') as f:
            texto = f.read()
            corpus_gutenberg.append(texto)
            nombres_gutenberg.append(archivo)
    except Exception as e:
        print(f' Error al leer {archivo}: {e}')

print(f'Libros cargados: {len(corpus_gutenberg)}')

```

Libros disponibles: 908
Libros cargados: 908

```

# Tokenizar el corpus Gutenberg
import time

print("Tokenizando libros...")
t0 = time.time()
corpus_guten_tokenizado = [tokenizar(doc) for doc in corpus_gutenberg]
t1 = time.time()
print(f"Tokenización completada en {t1-t0:.2f} s")

# Construir vocabulario (con max_features para limitar memoria)
vocab_g, vocab_lista_g = construir_vocabulario(
    corpus_guten_tokenizado, min_df=2, max_df_ratio=0.95
)

# Si el vocabulario es muy grande, limitar a las 50,000 palabras más frecuentes
if len(vocab_g) > 50000:
    # Recalcular frecuencia global y quedarse con las top 50,000
    freq_global = Counter()
    for doc_tokens in corpus_guten_tokenizado:
        freq_global.update(doc_tokens)

    top_palabras = set(w for w, _ in freq_global.most_common(50000) if w in vocab_g)
    vocab_lista_g = sorted(top_palabras)

```

```
vocab_g = {p: i for i, p in enumerate(vocab_lista_g)}
print(f"Vocabulario limitado a {len(vocab_g)} términos")
```

Tokenizando libros...

Tokenización completada en 137.18 s

Palabras únicas totales: 776199

Palabras después de filtrar (min_df=2, max_df=862): 295722

Palabras eliminadas por ser muy comunes (>862 docs): 703

Palabras eliminadas por ser muy raras (<2 docs): 479774

Vocabulario limitado a 48509 términos

```
# Construir matrices TF, DF, IDF, TF-IDF para Gutenberg
# Usamos sublinear_tf: log(1 + TF) en lugar de TF puro
```

```
print("Construyendo matriz TF...")
```

```
t0 = time.time()
```

```
matriz_tf_g = construir_matriz_tf(corpus_guten_tokenizado, vocab_g)
```

```
t1 = time.time()
```

```
print(f" Completado en {t1-t0:.2f} s")
```

```
# Aplicar sublinear TF: log(1 + TF) para documentos largos
```

```
print("Aplicando log(1 + TF)...")
```

```
matriz_tf_g = np.log1p(matriz_tf_g) # log(1 + x)
```

```
# Calcular DF e IDF
```

```
vector_df_g, vector_idf_g = construir_df_idf(matriz_tf_g)
```

```
# Calcular TF-IDF
```

```
matriz_tfidf_g = construir_matriz_tfidf(matriz_tf_g, vector_idf_g)
```

```
print(f"\nMatriz TF-IDF Gutenberg: {matriz_tfidf_g.shape}")
```

```
print(f" → {matriz_tfidf_g.shape[0]} libros × {matriz_tfidf_g.shape[1]} términos")
```

```
densidad = np.count_nonzero(matriz_tfidf_g) / (matriz_tfidf_g.shape[0] * matriz_tfidf_g.shape[1])
```

```
print(f"Densidad: {densidad:.2%}")
```

Construyendo matriz TF...

Completado en 13.36 s

Aplicando log(1 + TF)...

Matriz TF-IDF Gutenberg: (908, 48509)

→ 908 libros × 48509 términos

Densidad: 14.70%

```
# Términos más representativos de un libro específico
idx_libro = 0

vector_libro = matriz_tfidf_g[idx_libro]
top_indices = np.argsort(vector_libro)[::-1][:10]

print(f'Términos más representativos de "{nombres_gutenberg[idx_libro]}":')
for i in top_indices:
    print(f' {vocab_lista_g[i]:<25} TF-IDF = {vector_libro[i]:.4f}')
```

Términos más representativos de "20 poemas para ser leídos en el tranvía.txt":

oliverio	TF-IDF = 5.7839
vereda	TF-IDF = 4.3049
tranvía	TF-IDF = 4.2968
leídos	TF-IDF = 4.0073
pórfido	TF-IDF = 3.7897
nalgas	TF-IDF = 3.5727
croquis	TF-IDF = 3.4581
acueste	TF-IDF = 3.3009
ubres	TF-IDF = 3.2300
verona	TF-IDF = 3.1634

```
# Búsqueda en el corpus Gutenberg
consultas = [
    "amor y muerte",
    "historia de América",
    "aventura y viaje",
    "poesía lírica",
]

for consulta in consultas:
    print(f'\nConsulta: "{consulta}"')
    print('-' * 50)
    resultados = buscar(consulta, matriz_tfidf_g, vocab_g, vector_idf_g,
                        nombres=nombres_gutenberg, top_k=3)
    for libro, score in resultados:
        print(f' [{score:.4f}] {libro}')
```


Consulta: "amor y muerte"

[0.0071] Místicas poesías.txt
[0.0060] Lira Póstuma Obras Completas Vol. XXI.txt
[0.0055] El castigo sin venganza \$b Tragedia.txt

Consulta: "historia de América"

[0.0348] El derecho internacional americano estudio doctrinal y crítico.txt
[0.0219] Los Conquistadores El origen heroico de América.txt
[0.0216] España y los Estados Unidos de Norte América \$b a propósito de la guerra.txt

Consulta: "aventura y viaje"

[0.0218] Doble error \$b Novela.txt
[0.0183] Meditaciones del Quijote.txt
[0.0157] Vida de Don Quijote y Sancho.txt

Consulta: "poesía lírica"

[0.0469] El corazón juglar.txt
[0.0405] Libro de poemas.txt
[0.0395] Prosas Profanas Obras Completas Vol. II.txt

```
# Búsqueda interactiva
mi_consulta = "Don Quijote"

resultados = buscar(mi_consulta, matriz_tfidf_g, vocab_g, vector_idf_g,
                    nombres=nombres_gutenberg, top_k=5)

print(f'Resultados para: "{mi_consulta}"')
print('=' * 50)
for rank, (libro, score) in enumerate(resultados, start=1):
    print(f'{rank}. [{score:.4f}] {libro}')
```

Resultados para: "Don Quijote"

=====

1. [0.0498] El Consejo de los Dioses.txt
2. [0.0423] Vida de Cervantes.txt
3. [0.0417] Meditaciones del Quijote.txt
4. [0.0416] Vida de Don Quijote y Sancho.txt
5. [0.0318] Tres novelas ejemplares y un prólogo.txt